

THE EXPERT'S VOISE® IN JAVA

# Изучаем Java EE 7

Энтони Гонсалвес

apress®

 ПИТЕР®

# **Э. Гонсалвес**

## **Изучаем Java EE 7**

*Перевели с английского Е. Зазноба, О. Сивченко, О. Мясникова, В. Черник*

Заведующий редакцией  
Ведущий редактор  
Литературный редактор  
Художник  
Корректор  
Верстка

*Д. Виницкий  
Н. Гринчик  
В. Конаш  
Л. Адуевская  
Е. Павлович  
А. Барцевич*

ББК 32.973.2-018.1      УДК 004.43

**Гонсалвес Э.**

Г65 Изучаем Java EE 7. — СПб.: Питер, 2014. — 640 с.: ил.

ISBN 978-5-496-00942-3

Java Enterprise Edition (Java EE) остается одной из ведущих технологий и платформ на основе Java.

Данная книга представляет собой логичное пошаговое руководство, в котором подробно описаны многие спецификации и эталонные реализации Java EE 7. Работа с ними продемонстрирована на практических примерах. В этом фундаментальном издании также используется новейшая версия инструмента GlassFish, предназначенного для развертывания и администрирования примеров кода.

Книга написана ведущим специалистом по обработке запросов на спецификацию Java EE, членом наблюдательного совета организации Java Community Process (JCP). В ней вы найдете максимально ценную информацию, изложенную с точки зрения эксперта по технологиям Java для предприятий.

Благодаря этой книге вы познакомитесь с новейшей версией платформы Java EE; исследуете и научитесь использовать API EJB и JPA — от компонентов-сущностей, компонентов-сеансов до компонентов, управляемых сообщениями, и многого другого; откроете для себя API для разработки на веб-уровне, в частности JSF, Facelet и Expression Language; научитесь обращаться с веб-службами SOAP и RESTful, а также с другими службами, доступными в новейшей версии Java EE; узнаете, как создавать динамические пользовательские интерфейсы для корпоративных и транзакционных Java-приложений.

**12+** (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ISBN 978-1430246268 англ.  
ISBN 978-5-496-00942-3

Copyright © 2013 by Antonio Goncalves  
© Перевод на русский язык ООО Издательство «Питер», 2014  
© Издание на русском языке, оформление ООО Издательство «Питер», 2014

Права на издание получены по соглашению с Apress. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги.

ООО «Питер Пресс», 192102, Санкт-Петербург, ул. Андреевская (д. Волкова), д. 3, литер А, пом. 7Н.

Налоговая льгота — общероссийский классификатор продукции ОК 005-93, том 2; 95 3005 — литература учебная.

Подписано в печать 23.05.14. Формат 70×100/16. Усл. п. л. 51,600. Тираж 1000. Заказ 0000.

Отпечатано в полном соответствии с качеством предоставленных издательством материалов в ГППО «Псковская областная типография». 180004, Псков, ул. Ротная, 34.

# Оглавление

|                                                         |    |
|---------------------------------------------------------|----|
| <b>Предисловие</b> .....                                | 15 |
| <b>Об авторе</b> .....                                  | 16 |
| <b>О техническом редакторе</b> .....                    | 17 |
| <b>Благодарности</b> .....                              | 18 |
| <b>Введение</b> .....                                   | 19 |
| Структура книги .....                                   | 20 |
| Скачивание и запуск кода .....                          | 21 |
| Связь с автором .....                                   | 21 |
| От издательства .....                                   | 21 |
| <b>Глава 1. Краткий обзор Java EE 7</b> .....           | 22 |
| Понимание Java EE .....                                 | 22 |
| Архитектура .....                                       | 23 |
| Компоненты .....                                        | 24 |
| Контейнеры .....                                        | 25 |
| Сервисы .....                                           | 26 |
| Сетевые протоколы .....                                 | 27 |
| Упаковка .....                                          | 28 |
| Аннотации и дескрипторы развертывания .....             | 30 |
| Стандарты .....                                         | 32 |
| JCP .....                                               | 33 |
| Портируемость .....                                     | 33 |
| Модель программирования .....                           | 34 |
| Java Standard Edition 7 .....                           | 35 |
| Строковый оператор .....                                | 36 |
| Ромбовидная нотация .....                               | 36 |
| Конструкция try-with-resources .....                    | 37 |
| Multicatch-исключения .....                             | 38 |
| NIO.2 .....                                             | 38 |
| Обзор спецификаций Java EE .....                        | 39 |
| Краткая история Java EE .....                           | 39 |
| Отсечение .....                                         | 41 |
| Спецификации Java EE 7 .....                            | 42 |
| Спецификации веб-профиля 7 .....                        | 45 |
| Приложение CD-BookStore .....                           | 46 |
| Резюме .....                                            | 47 |
| <b>Глава 2. Контекст и внедрение зависимостей</b> ..... | 49 |
| Понятие компонентов .....                               | 49 |
| Внедрение зависимостей .....                            | 50 |
| Управление жизненным циклом .....                       | 51 |
| Области видимости и контекст .....                      | 52 |
| Перехват .....                                          | 52 |
| Слабая связанность и строгая типизация .....            | 53 |
| Дескриптор развертывания .....                          | 53 |
| Обзор спецификаций по CDI .....                         | 54 |
| Краткая история спецификаций CDI .....                  | 54 |

|                                                                                                      |           |
|------------------------------------------------------------------------------------------------------|-----------|
| Что нового в CDI 1.1 .....                                                                           | 55        |
| Базовая реализация .....                                                                             | 55        |
| Создание компонента CDI .....                                                                        | 55        |
| Внутренняя организация компонента CDI .....                                                          | 56        |
| Внедрение зависимостей .....                                                                         | 56        |
| Альтернативы .....                                                                                   | 64        |
| Производители данных .....                                                                           | 65        |
| Утилизаторы .....                                                                                    | 68        |
| Области видимости .....                                                                              | 69        |
| Компоненты в языке выражений .....                                                                   | 73        |
| Перехватчики .....                                                                                   | 73        |
| Перехватчики целевого класса .....                                                                   | 74        |
| Перехватчики классов .....                                                                           | 76        |
| Перехватчик жизненного цикла .....                                                                   | 78        |
| Связывание и исключение перехватчиков .....                                                          | 80        |
| Связывание с перехватчиком .....                                                                     | 80        |
| Приоритизация связывания перехватчиков .....                                                         | 82        |
| Декораторы .....                                                                                     | 83        |
| События .....                                                                                        | 84        |
| Все вместе .....                                                                                     | 87        |
| Написание классов Book и BookService .....                                                           | 88        |
| Написание классов NumberGenerator .....                                                              | 89        |
| Написание квалификаторов .....                                                                       | 90        |
| Написание автоматического журнала .....                                                              | 90        |
| Написание класса Main .....                                                                          | 91        |
| Приведение в действие CDI с beans.xml .....                                                          | 92        |
| Компиляция и выполнение с помощью Maven .....                                                        | 92        |
| Запуск класса Main .....                                                                             | 94        |
| Написание класса BookServiceIT .....                                                                 | 94        |
| Активизация альтернатив и перехватчиков в файлах beans.xml<br>для интеграционного тестирования ..... | 95        |
| Запуск интеграционного теста .....                                                                   | 95        |
| Резюме .....                                                                                         | 96        |
| <b>Глава 3. Валидация компонентов .....</b>                                                          | <b>97</b> |
| Понятие об ограничениях и валидации .....                                                            | 97        |
| Приложение .....                                                                                     | 98        |
| База данных .....                                                                                    | 99        |
| Клиент .....                                                                                         | 99        |
| Интероперабельность .....                                                                            | 100       |
| Обзор спецификации валидации компонентов .....                                                       | 100       |
| Краткая история валидации компонентов .....                                                          | 101       |
| Что нового появилось в версии Bean Validation 1.1 .....                                              | 101       |
| Справочная реализация .....                                                                          | 102       |
| Написание ограничений .....                                                                          | 102       |
| Внутренняя организация ограничения .....                                                             | 103       |
| Встроенные ограничения .....                                                                         | 106       |
| Определение собственных ограничений .....                                                            | 107       |
| Сообщения .....                                                                                      | 115       |
| Контекст ConstraintValidator .....                                                                   | 117       |
| Группы .....                                                                                         | 119       |
| Дескрипторы развертывания .....                                                                      | 120       |
| Валидация ограничений .....                                                                          | 121       |
| Валидационные API .....                                                                              | 122       |
| Валидация компонентов .....                                                                          | 125       |

|                                                              |            |
|--------------------------------------------------------------|------------|
| Валидация свойств .....                                      | 126        |
| Валидация значений .....                                     | 126        |
| Валидация методов .....                                      | 127        |
| Валидация групп .....                                        | 127        |
| Все вместе .....                                             | 128        |
| Написание компонента Customer .....                          | 130        |
| Написание компонента Address .....                           | 130        |
| Написание ограничения @Email .....                           | 131        |
| Написание ограничения @ZipCode .....                         | 131        |
| Написание интеграционных тестов CustomerIT и AddressIT ..... | 133        |
| Компиляция и тестирование в Maven .....                      | 134        |
| Резюме .....                                                 | 136        |
| <b>Глава 4. Java Persistence API .....</b>                   | <b>137</b> |
| Понятие сущностей .....                                      | 138        |
| Анатомия сущности .....                                      | 139        |
| Объектно-реляционное отображение .....                       | 140        |
| Выполнение запросов к сущностям .....                        | 143        |
| Единица сохраняемости .....                                  | 146        |
| Жизненный цикл сущности и обратные вызовы .....              | 146        |
| Интеграция с Bean Validation .....                           | 148        |
| Обзор спецификации JPA .....                                 | 148        |
| Краткая история JPA .....                                    | 149        |
| Что нового в JPA 2.1 .....                                   | 150        |
| Эталонная реализация .....                                   | 150        |
| Все вместе .....                                             | 151        |
| Написание сущности Book .....                                | 152        |
| Написание класса Main .....                                  | 152        |
| Написание интеграционного теста BookIT .....                 | 153        |
| Написание единицы сохраняемости .....                        | 155        |
| Написание SQL-сценария для загрузки данных .....             | 156        |
| Компиляция и тестирование с использованием Maven .....       | 157        |
| Применение класса Main с использованием Maven .....          | 160        |
| Проверка сгенерированной схемы .....                         | 161        |
| Резюме .....                                                 | 162        |
| <b>Глава 5. Объектно-реляционное отображение .....</b>       | <b>163</b> |
| Элементарное отображение .....                               | 163        |
| Таблицы .....                                                | 164        |
| Первичные ключи .....                                        | 166        |
| Атрибуты .....                                               | 171        |
| Тип доступа .....                                            | 177        |
| Коллекции базовых типов .....                                | 180        |
| Отображение базовых типов .....                              | 181        |
| Отображение с использованием XML .....                       | 183        |
| Встраиваемые объекты .....                                   | 186        |
| Отображение связей .....                                     | 189        |
| Связи в реляционных базах данных .....                       | 191        |
| Связи между сущностями .....                                 | 192        |
| Выборка связей .....                                         | 203        |
| Упорядочение связей .....                                    | 205        |
| Отображение наследования .....                               | 208        |
| Стратегии наследования .....                                 | 208        |
| Типы классов в иерархии наследования .....                   | 217        |
| Резюме .....                                                 | 221        |

|                                                                                |     |
|--------------------------------------------------------------------------------|-----|
| <b>Глава 6. Управление постоянными объектами</b> .....                         | 222 |
| Менеджер сущностей.....                                                        | 223 |
| Получение менеджера сущностей.....                                             | 225 |
| Контекст постоянства .....                                                     | 227 |
| Манипулирование сущностями.....                                                | 230 |
| JPQL.....                                                                      | 240 |
| SELECT .....                                                                   | 242 |
| FROM.....                                                                      | 244 |
| WHERE.....                                                                     | 244 |
| ORDER BY .....                                                                 | 245 |
| GROUP BY и HAVING .....                                                        | 246 |
| Массовое удаление .....                                                        | 246 |
| Массовое обновление .....                                                      | 247 |
| Запросы.....                                                                   | 247 |
| Динамические запросы .....                                                     | 250 |
| Именованные запросы.....                                                       | 252 |
| Criteria API (или объектно-ориентированные запросы).....                       | 254 |
| «Родные» запросы .....                                                         | 257 |
| Запросы к хранимым процедурам.....                                             | 258 |
| Cache API .....                                                                | 261 |
| Конкурентный доступ .....                                                      | 264 |
| Контроль версий.....                                                           | 266 |
| Оптимистическая блокировка .....                                               | 267 |
| Пессимистическая блокировка .....                                              | 269 |
| Жизненный цикл сущности.....                                                   | 270 |
| Обратные вызовы.....                                                           | 272 |
| Слушатели .....                                                                | 275 |
| Резюме .....                                                                   | 279 |
| <b>Глава 7. Корпоративные EJB-компоненты</b> .....                             | 281 |
| Понятие корпоративных EJB-компонентов .....                                    | 282 |
| Типы EJB-компонентов .....                                                     | 283 |
| Процесс и встроенный контейнер .....                                           | 284 |
| Службы, обеспечиваемые контейнером.....                                        | 285 |
| EJB Lite .....                                                                 | 286 |
| Обзор спецификации EJB .....                                                   | 287 |
| Краткая история спецификации EJB .....                                         | 288 |
| Что нового в EJB 3.2.....                                                      | 289 |
| Эталонная реализация.....                                                      | 290 |
| Написание корпоративных EJB-компонентов .....                                  | 290 |
| Анатомия EJB-компонента.....                                                   | 291 |
| Класс EJB-компонента.....                                                      | 292 |
| Удаленные и локальные представления, а также представление без интерфейса .... | 292 |
| Интерфейсы веб-служб .....                                                     | 295 |
| Переносимое JNDI-имя .....                                                     | 296 |
| EJB-компоненты без сохранения состояния .....                                  | 297 |
| EJB-компоненты с сохранением состояния .....                                   | 301 |
| Одиночные EJB-компоненты.....                                                  | 304 |
| Инициализация при запуске .....                                                | 306 |
| Объединение одиночных EJB-компонентов в цепочку.....                           | 306 |
| Конкурентный доступ .....                                                      | 307 |
| Конкурентный доступ, управляемый контейнером .....                             | 308 |
| Конкурентный доступ, управляемый EJB-компонентом .....                         | 309 |
| Время ожидания конкурентного доступа и запрет конкурентного доступа .....      | 310 |



|                                                                              |            |
|------------------------------------------------------------------------------|------------|
| Внедрение зависимостей.....                                                  | 311        |
| API-интерфейс SessionContext .....                                           | 312        |
| Асинхронные вызовы .....                                                     | 313        |
| Дескриптор развертывания .....                                               | 315        |
| Контекст именованной среды .....                                             | 317        |
| Упаковка .....                                                               | 318        |
| Развертывание EJB-компонента .....                                           | 319        |
| Вызов корпоративных EJB-компонентов .....                                    | 321        |
| Вызов с использованием внедрения .....                                       | 321        |
| Вызов с использованием CDI .....                                             | 322        |
| Вызов с использованием JNDI .....                                            | 323        |
| Резюме .....                                                                 | 324        |
| <b>Глава 8. Функции обратного вызова, служба таймера и авторизация .....</b> | <b>325</b> |
| Жизненный цикл сеансовых компонентов .....                                   | 325        |
| Компоненты, не сохраняющие состояние, и синглтоны .....                      | 326        |
| Компоненты, хранящие состояние .....                                         | 327        |
| Методы обратного вызова.....                                                 | 329        |
| Служба таймера .....                                                         | 332        |
| Выражения на основе календаря .....                                          | 333        |
| Декларативное создание таймера .....                                         | 335        |
| Программное создание таймера .....                                           | 336        |
| Авторизация.....                                                             | 338        |
| Декларативная авторизация .....                                              | 338        |
| Программная авторизация .....                                                | 341        |
| Все вместе .....                                                             | 343        |
| Написание сущности Entity .....                                              | 343        |
| Написание сеансового компонента BookEJB, не сохраняющего состояние .....     | 345        |
| Написание CDI DatabaseProducer.....                                          | 346        |
| Блок хранения для BookEJB .....                                              | 347        |
| Написание DatabasePopulator и определение источника данных.....              | 348        |
| Написание интеграционного теста BookEJBIT.....                               | 349        |
| Компиляция, тестирование и упаковка с помощью Maven .....                    | 351        |
| Развертывание на GlassFish .....                                             | 353        |
| Написание класса Main .....                                                  | 353        |
| Резюме .....                                                                 | 354        |
| <b>Глава 9. Транзакции .....</b>                                             | <b>355</b> |
| Понимание транзакций .....                                                   | 355        |
| Правила ACID.....                                                            | 356        |
| Условия считывания.....                                                      | 356        |
| Уровни изоляции транзакций.....                                              | 357        |
| Локальные транзакции JTA.....                                                | 357        |
| Распределенные транзакции и XA.....                                          | 358        |
| Обзор спецификаций для работы с транзакциями.....                            | 360        |
| Краткая история JTA .....                                                    | 361        |
| Что нового в версии 1.2 JTA .....                                            | 361        |
| Примеры реализации .....                                                     | 362        |
| Поддержка транзакций в EJB.....                                              | 362        |
| Маркировка CMT для отката транзакции .....                                   | 366        |
| Исключения и транзакции .....                                                | 367        |
| Поддержка транзакций в Managed Beans .....                                   | 371        |
| Резюме .....                                                                 | 373        |

|                                              |     |
|----------------------------------------------|-----|
| <b>Глава 10. JavaServer Faces</b> .....      | 374 |
| Концепция веб-страниц .....                  | 374 |
| HTML .....                                   | 375 |
| XHTML .....                                  | 377 |
| CSS .....                                    | 378 |
| DOM .....                                    | 381 |
| JavaScript .....                             | 381 |
| Концепция JSF .....                          | 384 |
| FacesServlet .....                           | 385 |
| Страницы и компоненты .....                  | 385 |
| Facelets .....                               | 387 |
| Отрисовщики .....                            | 387 |
| Преобразователи и валидаторы .....           | 387 |
| Компоненты-подложки и навигация .....        | 388 |
| Язык выражений .....                         | 389 |
| Поддержка AJAX .....                         | 390 |
| Обзор спецификации JSF .....                 | 390 |
| Краткая история веб-интерфейсов .....        | 390 |
| Краткая история JSF .....                    | 391 |
| Что нового в JSF 2.2 .....                   | 392 |
| Эталонные реализации .....                   | 393 |
| Создание JSF-страниц и компонентов .....     | 393 |
| Структура страницы JSF .....                 | 393 |
| Заголовок .....                              | 395 |
| Body .....                                   | 395 |
| Жизненный цикл .....                         | 398 |
| Анатомия компонентов JSF .....               | 400 |
| HTML-компоненты JSF .....                    | 400 |
| Команды .....                                | 401 |
| Цели .....                                   | 402 |
| Компоненты ввода .....                       | 402 |
| Компоненты вывода .....                      | 403 |
| Компоненты выбора .....                      | 404 |
| Графика .....                                | 406 |
| Сетка и таблицы .....                        | 406 |
| Сообщения об ошибках .....                   | 407 |
| Разное .....                                 | 408 |
| Базовые атрибуты .....                       | 409 |
| Основные теги JSF .....                      | 409 |
| Теги шаблонов JSF .....                      | 410 |
| Теги JSTL .....                              | 412 |
| Основные действия .....                      | 413 |
| Действия форматирования .....                | 414 |
| Управление ресурсами .....                   | 416 |
| Неявные объекты .....                        | 417 |
| Составные компоненты .....                   | 419 |
| Резюме .....                                 | 424 |
| <b>Глава 11. Обработка и навигация</b> ..... | 425 |
| Шаблон MVC .....                             | 425 |
| FacesServlet .....                           | 426 |
| FacesContext .....                           | 429 |
| Faces-config.xml .....                       | 430 |
| Создание компонентов-подложек .....          | 431 |



|                                                          |            |
|----------------------------------------------------------|------------|
| Структура компонентов-подложек.....                      | 432        |
| Области действия .....                                   | 434        |
| Жизненный цикл и аннотации функций обратного вызова..... | 435        |
| Обработка исключений и сообщений.....                    | 436        |
| Объединение JSF и EJB .....                              | 439        |
| Навигация .....                                          | 439        |
| Явная навигация .....                                    | 440        |
| Правила навигации .....                                  | 441        |
| Добавление страниц в закладки .....                      | 443        |
| Преобразование и проверка .....                          | 444        |
| Преобразователи .....                                    | 445        |
| Пользовательские преобразователи.....                    | 447        |
| Валидаторы .....                                         | 449        |
| Пользовательские валидаторы .....                        | 450        |
| Интеграция с Bean Validation .....                       | 451        |
| AJAX.....                                                | 451        |
| Общие понятия .....                                      | 452        |
| Поддержка в JSF.....                                     | 453        |
| Все вместе .....                                         | 455        |
| Написание сущности Book .....                            | 457        |
| Написание BookEJB.....                                   | 458        |
| Написание компонента-подложки BookController.....        | 458        |
| Написание шаблона layout.xhtml .....                     | 459        |
| Написание страницы newBook.xhtml .....                   | 460        |
| Написание страницы viewBook.xhtml.....                   | 462        |
| Компиляция и упаковка с помощью Maven .....              | 463        |
| Развертывание на GlassFish .....                         | 464        |
| Запуск примера .....                                     | 465        |
| Резюме .....                                             | 466        |
| <b>Глава 12. Обработка XML и JSON .....</b>              | <b>467</b> |
| Основные сведения об XML.....                            | 467        |
| XML-документ.....                                        | 468        |
| Проверка схемы XML .....                                 | 469        |
| Анализ с помощью SAX и DOM.....                          | 472        |
| DOM .....                                                | 472        |
| SAX .....                                                | 472        |
| Выполнение запросов с помощью XPath .....                | 473        |
| Преобразование с помощью XSLT .....                      | 473        |
| Обзор спецификаций XML.....                              | 475        |
| Краткая история XML-спецификаций .....                   | 475        |
| Спецификации XML в Java .....                            | 476        |
| Примеры реализаций .....                                 | 477        |
| Архитектура Java для обработки XML .....                 | 477        |
| Конфигурирование JAXP .....                              | 478        |
| JAXP и SAX.....                                          | 479        |
| JAXP и DOM .....                                         | 482        |
| JAXP и XSLT .....                                        | 483        |
| Архитектура Java для связывания XML .....                | 484        |
| Связывание .....                                         | 486        |
| Аннотации.....                                           | 487        |
| Основные сведения о JSON.....                            | 490        |
| Обзор спецификаций JSON.....                             | 492        |
| JSON-P .....                                             | 493        |
| Пример реализации.....                                   | 493        |

|                                                                  |            |
|------------------------------------------------------------------|------------|
| Обработка JSON .....                                             | 493        |
| Построение JSON .....                                            | 494        |
| Анализ JSON .....                                                | 495        |
| Генерация JSON .....                                             | 497        |
| Все вместе .....                                                 | 498        |
| Написание класса CreditCard .....                                | 499        |
| Написание теста CreditCardXMLTest .....                          | 499        |
| Написание теста CreditCardJSTest .....                           | 500        |
| Резюме .....                                                     | 501        |
| <b>Глава 13. Обмен сообщениями .....</b>                         | <b>502</b> |
| Основные сведения об обмене сообщениями .....                    | 502        |
| От точки к точке .....                                           | 504        |
| Публикация-подписка .....                                        | 505        |
| Администрируемые объекты .....                                   | 506        |
| Компоненты, управляемые сообщениями .....                        | 506        |
| Обзор спецификаций обмена сообщениями .....                      | 507        |
| Краткая история обмена сообщениями .....                         | 507        |
| Что нового в JMS 2.0 .....                                       | 508        |
| Что нового в EJB 3.2 .....                                       | 508        |
| Примеры реализации .....                                         | 508        |
| Java Messaging Service API .....                                 | 509        |
| Классический API .....                                           | 510        |
| ConnectionFactory .....                                          | 510        |
| Место назначения .....                                           | 511        |
| Соединение .....                                                 | 511        |
| Сеанс .....                                                      | 512        |
| Сообщения .....                                                  | 512        |
| Заголовок .....                                                  | 512        |
| Свойства .....                                                   | 514        |
| Тело сообщения .....                                             | 514        |
| Отправка и получение сообщений с помощью классического API ..... | 514        |
| Упрощенный API .....                                             | 516        |
| JMSContext .....                                                 | 517        |
| JMSProducer .....                                                | 519        |
| JMSConsumer .....                                                | 519        |
| Написание производителей сообщений .....                         | 520        |
| Производство сообщения вне контейнера .....                      | 520        |
| Производство сообщения внутри контейнера .....                   | 521        |
| Производство сообщений внутри контейнера с помощью CDI .....     | 522        |
| Написание потребителей сообщений .....                           | 523        |
| Синхронная доставка .....                                        | 523        |
| Асинхронная доставка .....                                       | 525        |
| Механизмы надежности .....                                       | 526        |
| Фильтрация сообщений .....                                       | 527        |
| Настройка параметров времени существования сообщений .....       | 527        |
| Задание стойкости сообщения .....                                | 528        |
| Управление подтверждением .....                                  | 528        |
| Создание стойких потребителей .....                              | 529        |
| Определение приоритетов .....                                    | 529        |
| Написание компонентов, управляемых сообщениями .....             | 529        |
| Структура MDB .....                                              | 530        |
| @MessageDriven .....                                             | 531        |
| @ActivationConfigProperty .....                                  | 532        |
| Внедрение зависимостей .....                                     | 533        |

|                                                           |            |
|-----------------------------------------------------------|------------|
| Контекст MDB .....                                        | 533        |
| Жизненный цикл и аннотации функций обратного вызова ..... | 534        |
| MDB как потребитель .....                                 | 535        |
| MDB как производитель сообщений .....                     | 535        |
| Транзакции .....                                          | 536        |
| Обработка исключений .....                                | 537        |
| Все вместе .....                                          | 538        |
| Написание класса OrderDTO .....                           | 538        |
| Написание класса OrderProducer .....                      | 539        |
| Написание OrderConsumer .....                             | 540        |
| Написание класса ExpensiveOrderMDB .....                  | 540        |
| Компиляция и упаковка с помощью Maven .....               | 541        |
| Создание администрируемых объектов .....                  | 542        |
| Развертывание MDB на GlassFish .....                      | 543        |
| Выполнение примера .....                                  | 543        |
| Резюме .....                                              | 544        |
| <b>Глава 14. Веб-службы SOAP .....</b>                    | <b>545</b> |
| Основные сведения о веб-службах SOAP .....                | 546        |
| XML .....                                                 | 547        |
| WSDL .....                                                | 547        |
| SOAP .....                                                | 550        |
| UDDI .....                                                | 551        |
| Транспортный протокол .....                               | 552        |
| Обзор спецификаций веб-служб SOAP .....                   | 552        |
| Краткая история спецификаций веб-служб SOAP .....         | 552        |
| Спецификации, связанные с веб-службами SOAP .....         | 553        |
| JAX-WS 2.2a .....                                         | 554        |
| Web Services 1.3 .....                                    | 554        |
| WS-Metadata 2.3 .....                                     | 555        |
| Что нового в спецификации веб-служб SOAP .....            | 555        |
| Примеры реализаций .....                                  | 555        |
| Создание веб-служб SOAP .....                             | 555        |
| Структура веб-службы SOAP .....                           | 557        |
| Конечные точки веб-служб SOAP .....                       | 558        |
| Преобразование WSDL .....                                 | 558        |
| @WebService .....                                         | 559        |
| @WebMethod .....                                          | 560        |
| @WebResult .....                                          | 561        |
| @WebParam .....                                           | 562        |
| @OneWay .....                                             | 562        |
| @SoapBinding .....                                        | 562        |
| Собираем все преобразования воедино .....                 | 564        |
| Обработка исключений .....                                | 567        |
| Жизненный цикл и функции обратного вызова .....           | 570        |
| WebServiceContext .....                                   | 570        |
| Дескриптор развертывания .....                            | 571        |
| Упаковка .....                                            | 572        |
| Публикация веб-служб SOAP .....                           | 572        |
| Вызов веб-служб SOAP .....                                | 573        |
| Структура потребителя SOAP .....                          | 574        |
| Программный вызов .....                                   | 574        |
| Вызов с помощью внедрения .....                           | 575        |
| Вызов с помощью CDI .....                                 | 576        |
| Все вместе .....                                          | 577        |

|                                                                        |            |
|------------------------------------------------------------------------|------------|
| Написание класса CreditCard .....                                      | 577        |
| Написание веб-службы SOAP CardValidator .....                          | 579        |
| Написание модульного теста CardValidatorTest .....                     | 579        |
| Написание интеграционного теста CardValidatorIT .....                  | 580        |
| Компиляция, тестирование и упаковка с помощью Maven .....              | 581        |
| Развертывание на GlassFish .....                                       | 583        |
| Написание класса WebServiceConsumer .....                              | 584        |
| Создание артефактов потребителя и упаковка с помощью Maven .....       | 585        |
| Запуск класса WebServiceConsumer .....                                 | 588        |
| Резюме .....                                                           | 588        |
| <b>Глава 15. Веб-службы в стиле REST .....</b>                         | <b>589</b> |
| Понятие о веб-службах RESTful .....                                    | 590        |
| Практика работы в браузере .....                                       | 590        |
| Ресурсы и URI .....                                                    | 590        |
| Представления .....                                                    | 592        |
| Адресуемость .....                                                     | 592        |
| Связность .....                                                        | 593        |
| Единообразный интерфейс .....                                          | 594        |
| Отсутствие сохранения состояния .....                                  | 594        |
| HTTP .....                                                             | 595        |
| От Сети к веб-службам .....                                            | 602        |
| WADL .....                                                             | 602        |
| Обзор спецификаций веб-служб с передачей состояния представления ..... | 603        |
| Краткая история REST .....                                             | 603        |
| API Java для веб-служб с передачей состояния представления .....       | 604        |
| Что нового в JAX-RS 2.0 .....                                          | 604        |
| Справочная реализация .....                                            | 605        |
| Написание веб-служб с передачей состояния представления .....          | 605        |
| Структура веб-службы с передачей состояния представления .....         | 606        |
| Выполнение операций CRUD над веб-службами в стиле REST .....           | 606        |
| Определение URI и URI связывания .....                                 | 608        |
| Извлечение параметров .....                                            | 609        |
| Использование и создание типов содержимого .....                       | 611        |
| Возвращаемые типы .....                                                | 613        |
| Сопоставление HTTP-методов .....                                       | 615        |
| Контекстная информация .....                                           | 617        |
| Поставщик объектов .....                                               | 619        |
| Обработка исключений .....                                             | 621        |
| Жизненный цикл и обратные вызовы .....                                 | 623        |
| Упаковка .....                                                         | 623        |
| Вызов веб-служб в стиле REST .....                                     | 624        |
| Клиентский API .....                                                   | 624        |
| Структура потребителя REST .....                                       | 627        |
| Все вместе .....                                                       | 628        |
| Написание сущности Book .....                                          | 628        |
| Написание JAXB-компонента Books .....                                  | 629        |
| Написание службы BookRestService .....                                 | 630        |
| Конфигурирование JAX-RS .....                                          | 634        |
| Компиляция и упаковка с помощью Maven .....                            | 635        |
| Развертывание в GlassFish .....                                        | 637        |
| WADL .....                                                             | 637        |
| Написание интеграционного теста BookRestServiceIT .....                | 638        |
| Резюме .....                                                           | 639        |

# Предисловие

Java EE 7 основана на предыдущих успешных версиях этой платформы. Упрощенный API сервиса сообщений Java значительно повышает продуктивность разработчика, широкое использование контекстов и внедрения зависимостей (CDI) и сокращение шаблонного кода обеспечивают более связанную интегрированную платформу. Java EE 7 также включает такие технологии, как WebSocket, JSON, Batch и Concurrency, весьма существенные для современной разработки веб-приложений. Все это сокращает необходимость в сторонних фреймворках, значительно облегчая приложения.

Энтони сыграл очень важную роль в формировании платформы Java EE 7. Благодаря своим глубоким техническим знаниям он принял активное участие в составлении спецификаций во время двух ключевых запросов на спецификацию Java (платформа и Enterprise JavaBeans 3.2). В документах он выделил несколько пунктов, что сделало их более простыми для понимания. Он также участвовал в разъяснительных совещаниях при поддержке комитета Java Community Process (JCP), а также входил в состав данного комитета.

Энтони возглавляет парижскую группу пользователей Java; работая консультантом, он использует Java EE для решения повседневных проблем. С энтузиазмом и преданностью своему делу он проводит конференцию DevOxx France. Кроме того, несколько лет назад он написал книгу о Java EE 5 на французском языке, а после — получившую высокое признание книгу «Введение в платформу Java EE 6 с GlassFish 3» (*Beginning Java EE 6 Platform with GlassFish 3*). Благодаря этому Энтони стал наилучшим кандидатом на авторство данной книги.

Книга содержит несколько практических примеров кода, с которых можно начать изучение. Для развертывания примеров используется GlassFish, но также подойдет любой сервер приложений, совместимый с Java EE 7. Все представленные образцы кода также доступны на GitHub. Если вы искали практический учебник, написанный одним из экспертов по этой платформе, названный источник вам подойдет.

*Арун Гунта,  
специалист по Java EE и GlassFish*

# Введение

В сегодняшнем мире бизнеса приложения должны осуществлять доступ к данным, реализовывать бизнес-логику, добавлять уровни представления данных, быть мобильными, использовать геолокацию и взаимодействовать с внешними системами и онлайн-сервисами. Именно этого пытаются достичь компании путем минимизации затрат и применения стандартных и надежных технологий, которые могут справиться с большими нагрузками. Если это ваш случай, данная книга — для вас.

Среда Java Enterprise Edition появилась в конце 1990-х годов и привнесла в язык Java надежную программную платформу для разработок корпоративного уровня. J2 EE, хоть и составляла конкуренцию фреймворкам с открытым кодом, считалась достаточно тяжеловесной технологией, была технически перенасыщенной, и каждая ее новая версия подвергалась критике, неправильно понималась или использовалась. Благодаря этой критике Java EE была усовершенствована и упрощена.

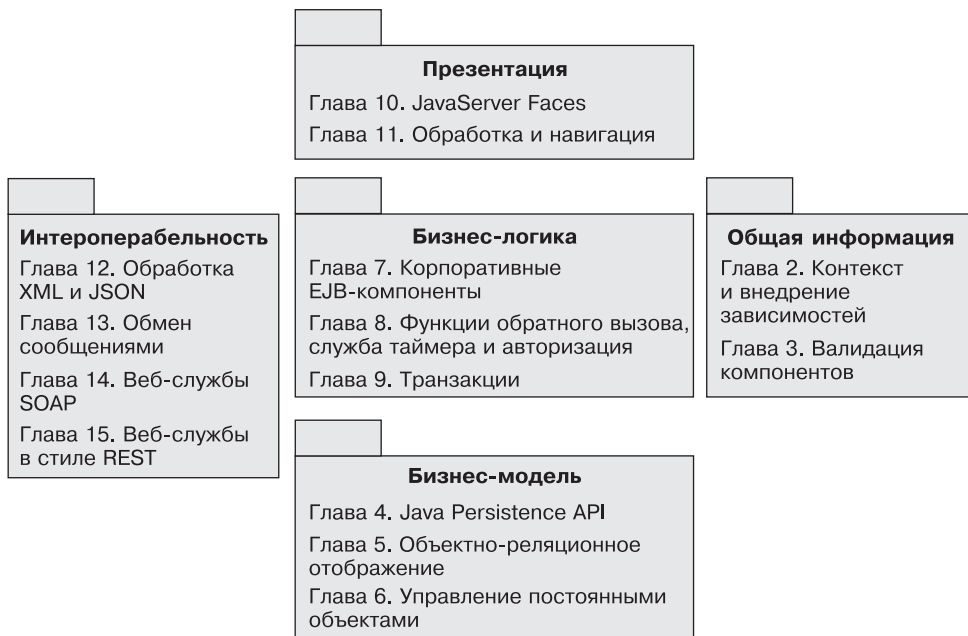
Если вы относитесь к числу людей, которые по-прежнему считают, что «архитектуры EJB — это зло», прочитайте эту книгу, и вы измените свое мнение. Архитектуры Enterprise Java Beans просто прекрасны, как и весь стек технологий Java EE 7. Если же вы, наоборот, адепт Java EE, то в книге вы увидите, как эта платформа обрела равновесие благодаря простоте разработки и несложной компонентной модели. Если вы начинающий пользователь Java EE, эта книга вам также подойдет: в ней очень понятно описываются наиболее важные спецификации, а также для наглядности приводится много примеров кода и схем.

Открытые стандарты являются в совокупности одной из наиболее сильных сторон Java EE. Теперь не составляет труда портировать прикладные программы между серверами приложений. Диапазон технологий, применяемых при написании этих программ, включает JPA, CDI, валидацию компонентов (Bean Validation), EJB, JSF, JMS, SOAP веб-службы либо веб-службы с передачей состояния представления (RESTful). Открытый исходный код — еще одна сильная сторона Java EE. Как вы увидите далее в книге, большинство базовых реализаций Java EE 7 (GlassFish, EclipseLink, Weld, Hibernate Validator, Mojarra, OpenMQ, Metro и Jersey) лицензируются как свободно распространяемое ПО.

Книга посвящена инновациям последней версии, различным спецификациям и способам их сочетания при разработке приложений. Java EE 7 включает около 30 спецификаций и является важным этапом разработок для корпоративного уровня (CDI 1.1, Bean Validation 1.1, EJB 3.2, JPA 2.1), для веб-уровня (Servlet 3.1, JSF 2.2, Expression Language 3.0) и для интероперабельности (JAX-WS 2.3 и JAX-RS 2.0). Издание охватывает большую часть спецификаций по Java EE 7 и использует инструментарий JDK 1.7 и некоторые известные шаблоны проектирования, а также сервер приложений GlassFish, базу данных Derby, библиотеку JUnit и фреймворк Maven. Издание изобилует UML-схемами, примерами Java-кода и скриншотами.

## Структура книги

Книга посвящена наиболее важным спецификациям по Java EE 7 и новому функционалу этого релиза. Структура издания повторяет архитектурное выделение уровней в приложении.



В главе 1 приводятся основные понятия Java EE 7, которые далее обсуждаются в книге. Глава 2 посвящена контексту и внедрению зависимости (Context and Dependency Injection 1.1), а глава 3 — валидации компонентов (Bean Validation 1.1).

Главы 4–6 описывают уровень сохраняемости и подробно рассматривают интерфейс JPA 2.1. После краткого обзора и нескольких практических примеров в главе 4 глава 5 полностью посвящена объектно-реляционному отображению (атрибутам отображения, отношениям и наследованию), а глава 6 — тому, как вызывать сущности и управлять ими, их жизненному циклу, методам обратного вызова и слушателям.

Для разработки транзакций на уровне бизнес-логики на Java EE 7 может использоваться архитектура EJB. Главы 7–9 описывают этот процесс. После обзора спецификации и истории ее создания в главе 7 идет речь о компонент-сеансах EJB и модели их разработки. Глава 8 посвящена жизненному циклу архитектуры EJB, службе времени и способам обращения с авторизацией. Глава 9 объясняет понятие транзакций и то, каким образом интерфейс JTA переводит транзакции в EJB, а также в CDI-компоненты.

В главах 10 и 11 описано, как ведется разработка уровня представления данных с помощью фреймворка JSF 2.2. После обзора спецификации в главе 10 речь идет



о разработке веб-страницы с помощью компонентов фреймворков JSF и Facelets. В главе 11 рассказывается о способах взаимодействия с серверной частью архитектуры EJB и поддерживающими CDI-компонентами, а также о навигации по страницам.

Наконец, последние главы предлагают различные способы взаимодействия с другими системами. В главе 12 объясняется, как обрабатывать XML (используя интерфейсы JAXB и JAXP) и JSON (JSON-P 1.0). Глава 13 показывает, как обмениваться асинхронными сообщениями с новым интерфейсом JMS 2.0 и компонентами, управляемыми сообщениями. Глава 14 посвящена веб-службам SOAP, а глава 15 — веб-службам RESTful с новым интерфейсом JAX-RS 2.0.

## Скачивание и запуск кода

Для создания примеров, приводимых в этой книге, использовался фреймворк Maven 3 и комплект Java-разработчика JDK 1.7 в качестве компилятора. Развертывание осуществлялось на сервере приложений GlassFish версии 4, а сохранение данных — в базе Derby. В каждой из глав объясняется, как построить, развернуть, запустить и протестировать компоненты в зависимости от используемой технологии. Код тестировался на платформе Mac OS X (но также должен работать в Windows или Linux). Исходный код примеров к этой книге размещен на странице Source Code на сайте Apress ([www.apress.com](http://www.apress.com)). Загрузить его можно прямо с GitHub по адресу <https://github.com/agoncal/agoncal-book-Java EE7>.

## Связь с автором

Вопросы по содержанию этой книги, коду либо другим темам отправляйте автору на электронную почту [antonio.goncalves@gmail.com](mailto:antonio.goncalves@gmail.com). Вы также можете посетить его сайт [www.antoniongoncalves.org](http://www.antoniongoncalves.org) и подписаться на его страницу в «Твиттере»: @agoncal.

## От издательства

Ваши замечания, предложения и вопросы отправляйте по адресу электронной почты [vinitski@minsk.piter.com](mailto:vinitski@minsk.piter.com) (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На сайте издательства <http://www.piter.com> вы найдете подробную информацию о наших книгах.

# Глава 1

## Краткий обзор Java EE 7

Сегодняшние предприятия существуют в условиях мировой конкуренции. Им нужны приложения, отвечающие их производственным нуждам, которые, в свою очередь, с каждым днем усложняются. В эпоху глобализации компании могут действовать по всему миру, имея представительства на разных континентах, работать в различных странах круглосуточно, без выходных, иметь по несколько центров обработки данных и международные системы, работающие с разными валютами и временными зонами. При этом им необходимо сокращать расходы, увеличивать быстродействие своих сервисов, хранить бизнес-данные в надежных и безопасных хранилищах, а также иметь несколько мобильных и веб-интерфейсов для клиентов, сотрудников и поставщиков.

Большинству компаний необходимо совмещать эти противоречивые требования с существующими корпоративными информационными системами (EIS), одновременно разрабатывая приложения типа «бизнес — бизнес» для работы с партнерами или системы типа «бизнес — потребитель» с использованием мобильных приложений, в том числе с возможностью геолокации. Довольно часто компании требуется координировать корпоративные данные, которые хранятся в разных местах, обрабатываются несколькими языками программирования и передаются с помощью разных протоколов. И конечно же, во избежание серьезных убытков необходимо предотвращать системные сбои, сохраняя при этом высокую доступность, масштабируемость и безопасность. Изменяясь и усложняясь, корпоративные приложения должны оставаться надежными. Именно для этого была создана платформа Java Enterprise Edition (Java EE).

Первая версия Java EE (изначально известная как J2EE) предназначалась для решения задач, с которыми сталкивались компании в 1999 году, а именно для работы с распределенными компонентами. С тех пор программным приложениям пришлось адаптироваться к новым техническим решениям, таким как веб-службы SOAP и RESTful. На сегодняшний день платформа Java EE отвечает этим техническим требованиям, регламентируя различные способы работы в стандартных спецификациях. Спустя годы Java EE изменилась, стала насыщеннее и проще в использовании, а также более мобильной и интегрированной.

В этой главе будут представлены общие сведения о Java EE. После описания внутренней архитектуры, компонентов и сервисов я расскажу о том, что нового в Java EE 7.

### Понимание Java EE

Если вам нужно произвести какие-то операции с наборами объектов, вы не разрабатываете для этого свою собственную хеш-таблицу; вы используете API кол-

лекций (интерфейс программирования приложений). Точно так же, если вам требуется простое веб-приложение или безопасная, интероперабельная распределенная прикладная система, оперирующая транзакциями, вы не захотите разрабатывать низкоуровневые API-интерфейсы, а будете использовать платформу Java EE. Так же как платформа Java Standard Edition (Java SE) предоставляет API для работы с коллекциями, Java EE предоставляет стандартный способ работы с транзакциями через Java API для транзакций (JTA), с сообщениями через службу сообщений Java (JMS) и с сохраняемостью через интерфейс JPA. Java EE располагает рядом спецификаций, предназначенных для корпоративных приложений. Она может рассматриваться как продолжение платформы Java SE для более удобной разработки распределенных, надежных, мощных и высокодоступных приложений.

Версия Java EE 7 стала важной вехой в развитии платформы. Она не только продолжает традиции Java EE 6, предлагая более простую модель разработки, но и добавляет свежие спецификации, обогащая наработанный функционал новыми возможностями. Кроме того, контекст и внедрение зависимостей (CDI) становится точкой интеграции между всеми новыми спецификациями. Релиз Java EE 7 почти совпадает с 13-й годовщиной выпуска корпоративной платформы. Она объединяет преимущества языка Java и опыт последних 13 лет. Java EE выигрывает как за счет динамизма сообществ свободных разработчиков, так и за счет строгой стандартизации группы Java Community Process (JCP). На сегодняшний день Java EE — это хорошо документированная платформа с опытными разработчиками, большим сообществом пользователей и множеством развертываемых приложений, работающих на серверах компаний. Java EE объединяет несколько интерфейсов API, которые могут использоваться для построения стандартных компонентно-ориентированных многозвенных приложений. Эти компоненты развертываются в различных контейнерах, предлагая серию служб.

## Архитектура

Java EE состоит из набора спецификаций, реализуемых различными контейнерами. Контейнерами называются средства среды времени выполнения Java EE, предоставляющие размещенным на них компонентам определенные службы, например управление жизненным циклом разработки, внедрение зависимости, параллельный доступ и т. д. Такие компоненты используют точно определенные контракты для сообщения с инфраструктурой Java EE и с другими компонентами. Перед развертыванием они должны упаковываться стандартным способом (повторяя структуру определенного каталога, который может быть сжат в архивный файл). Java EE представляет собой расширенный набор функций платформы Java SE, что означает, что API-интерфейсы Java SE могут использоваться любыми компонентами Java EE.

Рисунок 1.1 иллюстрирует логические взаимосвязи между контейнерами. Стрелками представлены протоколы, используемые одним контейнером для доступа к другому. Например, веб-контейнер размещает сервлеты, которые могут обращаться к компонентам EJB по протоколу RMI-IIOP.

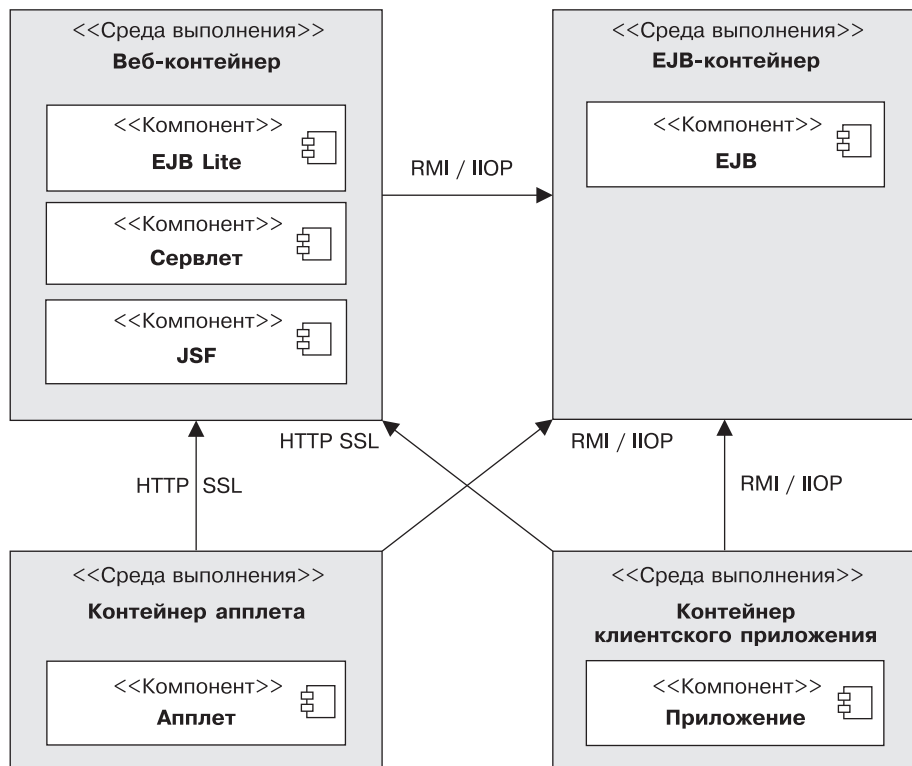


Рис. 1.1. Стандартные контейнеры Java EE

## Компоненты

В среде времени выполнения Java EE выделяют четыре типа компонентов, которые должна поддерживать реализация.

- ❑ *Апплеты* представляют собой приложения из графического пользовательского интерфейса (GUI), выполняемые в браузере. Они задействуют насыщенный интерфейс Swing API для производства мощных пользовательских интерфейсов.
- ❑ *Приложениями* называются программы, выполняемые на клиентской стороне. Как правило, они относятся к графическому пользовательскому интерфейсу (GUI) и применяются для пакетной обработки. Приложения имеют доступ ко всем средствам среднего звена.
- ❑ *Веб-приложения* (состоят из сервлетов и их фильтров, слушателей веб-событий, страниц JSP и JSF) выполняются в веб-контейнере и отвечают на запросы HTTP от веб-клиентов. Сервлеты также поддерживают конечные точки веб-служб SOAP и RESTful. Веб-приложения также могут содержать компоненты EJB Lite (подробнее об этом читайте в гл. 7).
- ❑ *Корпоративные приложения* (созданные с помощью технологии Enterprise Java Beans, службы сообщений Java Message Service, интерфейса Java API для транзакций, асинхронных вызовов, службы времени, протоколов RMI-IIOP) вы-

полняются в контейнере EJB. Управляемые контейнером компоненты EJB служат для обработки транзакционной бизнес-логики. Доступ к ним может быть как локальным, так и удаленным по протоколу RMI (или HTTP для веб-служб SOAP и RESTful).

## Контейнеры

Инфраструктура Java EE подразделяется на логические домены, называемые контейнерами (см. рис. 1.1). Каждый из контейнеров играет свою специфическую роль, поддерживает набор интерфейсов API и предлагает компонентам сервисы (безопасность, доступ к базе данных, обработку транзакций, присваивание имен каталогам, внедрение ресурсов). Контейнеры скрывают техническую сложность и повышают мобильность. При разработке приложений каждого типа необходимо учитывать возможности и ограничения каждого контейнера, чтобы знать, использовать один или несколько. Например, для разработки веб-приложения необходимо сначала разработать уровень фреймворка JSF и уровень EJB Lite, а затем развернуть их в веб-контейнер. Но если вы хотите, чтобы веб-приложение удаленно вызывало бизнес-уровень, а также использовало передачу сообщений и асинхронные вызовы, вам потребуется как веб-, так и EJB-контейнер.

Java EE использует четыре различных контейнера.

- ❑ *Контейнеры апплетов* выполняются большинством браузеров. При разработке апплетов можно сконцентрироваться на визуальной стороне приложения, в то время как контейнер обеспечивает безопасную среду. Контейнер апплета использует модель безопасности изолированной программной среды («песочницы»), где коду, выполняемому в «песочнице», не разрешается «играть» за ее пределами. Это означает, что контейнер препятствует любому коду, загруженному на ваш локальный компьютер, получать доступ к локальным ресурсам системы (процессам либо файлам).
- ❑ *Контейнер клиентского приложения (ACC)* включает набор Java-классов, библиотек и других файлов, необходимых для реализации в приложениях Java SE таких возможностей, как внедрение, управление безопасностью и служба именования (в частности, Swing, пакетная обработка либо просто класс с методом `main()`). Контейнер ACC обращается к EJB-контейнеру, используя протокол RMI-IIOP, а к веб-контейнеру — по протоколу HTTP (например, для веб-служб).
- ❑ *Веб-контейнер* предоставляет базовые службы для управления и исполнения веб-компонентов (сервлетов, компонентов EJB Lite, страниц JSP, фильтров, слушателей, страниц JSF и веб-служб). Он отвечает за создание экземпляров, инициализацию и вызов сервлетов, а также поддержку протоколов HTTP и HTTPS. Этот контейнер используется для подачи веб-страниц к клиент-браузерам.
- ❑ *EJB-контейнер* отвечает за управление и исполнение компонентов модели EJB (компонент-сесансы EJB и компоненты, управляемые сообщениями), содержащих уровень бизнес-логики вашего приложения на Java EE. Он создает новые сущности компонентов EJB, управляет их жизненным циклом и обеспечивает реализацию таких сервисов, как транзакция, безопасность, параллельный доступ, распределение, служба именования либо возможность асинхронного вызова.

## Сервисы

Контейнеры развертывают свои компоненты, предоставляя им соответствующие базовые сервисы. Контейнеры позволяют разработчику сконцентрироваться на реализации бизнес-логики, а не решать технические проблемы, присутствующие в корпоративных приложениях. На рис. 1.2 изображены сервисы, предлагаемые каждым контейнером. Например, веб- и EJB-контейнеры предоставляют коннекторы для доступа к информационной системе предприятия, но не к контейнеру апплетов или контейнеру клиентского приложения. Java EE предлагает следующие сервисы.

- ❑ *Java API для транзакций* — этот сервис предлагает интерфейс разграничения транзакций, используемый контейнером и приложением. Он также предоставляет интерфейс между диспетчером транзакций и диспетчером ресурсов на уровне интерфейса драйвера службы.
- ❑ *Интерфейс сохраняемости Java* — стандартный интерфейс для объектно-реляционного отображения (ORM). С помощью встроенного языка запросов JPQL вы можете обращаться к объектам, хранящимся в основной базе данных.
- ❑ *Валидация* — благодаря валидации компонентов объявляется ограничение целостности на уровне класса и метода.
- ❑ *Интерфейс Java для доступа к службам сообщений* — позволяет компонентам асинхронно обмениваться данными через сообщения. Он поддерживает надежный обмен сообщениями по принципу «от точки к точке» (P2P), а также модель публикации-подписки (pub-sub).
- ❑ *Java-интерфейс каталогов и служб именованя (JNDI)* — этот интерфейс, появившийся в Java SE, используется как раз для доступа к системам служб именованя и каталогов. Ваше приложение применяет его, чтобы ассоциировать (связывать) имена с объектами и затем находить их в каталогах. Вы можете задать поиск источников данных, фабрик классов JMS, компонентов EJB и других ресурсов. Интерфейс JNDI, повсеместно присутствовавший в коде до версии 1.4 J2EE, в настоящее время используется более прозрачным способом — через внедрение.
- ❑ *Интерфейс JavaMail* — многим приложениям требуется функция отправки сообщений электронной почты, которая может быть реализована благодаря этому интерфейсу.
- ❑ *Фреймворк активизации компонентов JavaBeans (JAF)* — интерфейс JAF, являющийся составной частью платформы Java SE, предоставляет фреймворк для обработки данных различных MIME-типов. Используется сервисом JavaMail.
- ❑ *Обработка XML* — большинство компонентов Java EE могут развертываться с помощью опциональных дескрипторов развертывания XML, а приложениям часто приходится манипулировать XML-документами. Интерфейс Java для обработки XML (JAXP) поддерживает синтаксический анализ документов с применением интерфейсов SAX и DOM, а также на языке XSLT.
- ❑ *Обработка JSON (объектной нотации JavaScript)* — появившийся только в Java EE 7 Java-интерфейс для обработки JSON (JSON-P) позволяет приложениям

синтаксически анализировать, генерировать, трансформировать и запрашивать JSON.

- ❑ *Архитектура коннектора Java EE* — коннекторы позволяют получить доступ к корпоративным информационным системам (EIS) с компонента Java EE. К таким компонентам относятся базы данных, мейнфреймы либо программы для планирования и управления ресурсами предприятия (ERP).
- ❑ *Службы безопасности* — служба аутентификации и авторизации для платформы Java (JAAS) позволяет сервисам аутентифицироваться и устанавливать права доступа, обязательные для пользователей. Контракт поставщика сервиса авторизации Java для контейнеров (JACC) определяет соглашение о взаимодействии между сервером приложений Java EE и поставщиком сервиса авторизации, позволяя, таким образом, сторонним поставщикам такого сервиса подключаться к любому продукту Java EE. Интерфейс поставщика сервисов аутентификации Java для контейнеров (JASPIC) определяет стандартный интерфейс, с помощью которого модули аутентификации могут быть интегрированы с контейнерами. В результате модули могут установить идентификаторы подлинности, используемые контейнерами.
- ❑ *Веб-службы* — Java EE поддерживает веб-службы SOAP и RESTful. Интерфейс Java для веб-служб на XML (JAX-WS), сменивший интерфейс Java с поддержкой вызовов удаленных процедур на основе XML (JAX-RPC), обеспечивает работу веб-служб, работающих по протоколу SOAP/HTTP. Интерфейс Java для веб-служб RESTful (JAX-RS) поддерживает веб-службы, использующие стиль REST.
- ❑ *Внедрение зависимостей* — начиная с Java EE 5, некоторые ресурсы могут внедряться в управляемые компоненты. К таким ресурсам относятся источники данных, фабрики классов JMS, единицы сохраняемости, компоненты EJB и т. д. Кроме того, для этих целей Java EE 7 использует спецификации по контексту и внедрению зависимости (CDI), а также внедрение зависимости для Java (DI).
- ❑ *Управление* — Java EE с помощью специального управляющего компонента определяет API для операций с контейнерами и серверами. Интерфейс управляющих расширений Java (JMXAPI) также используется для поддержки управления.
- ❑ *Развертывание* — спецификация Java EE по развертыванию определяет соглашение о взаимодействии между средствами развертывания и продуктами Java EE для стандартизации развертывания приложения.

## Сетевые протоколы

Как показано на рис. 1.2, компоненты, развертываемые в контейнерах, могут вызываться с помощью различных протоколов. Например, сервлет, развертываемый в веб-контейнере, может вызываться по протоколу HTTP, как и веб-служба с конечной точкой EJB, развертываемый в EJB-контейнере. Ниже приводится список протоколов, поддерживаемых Java EE.

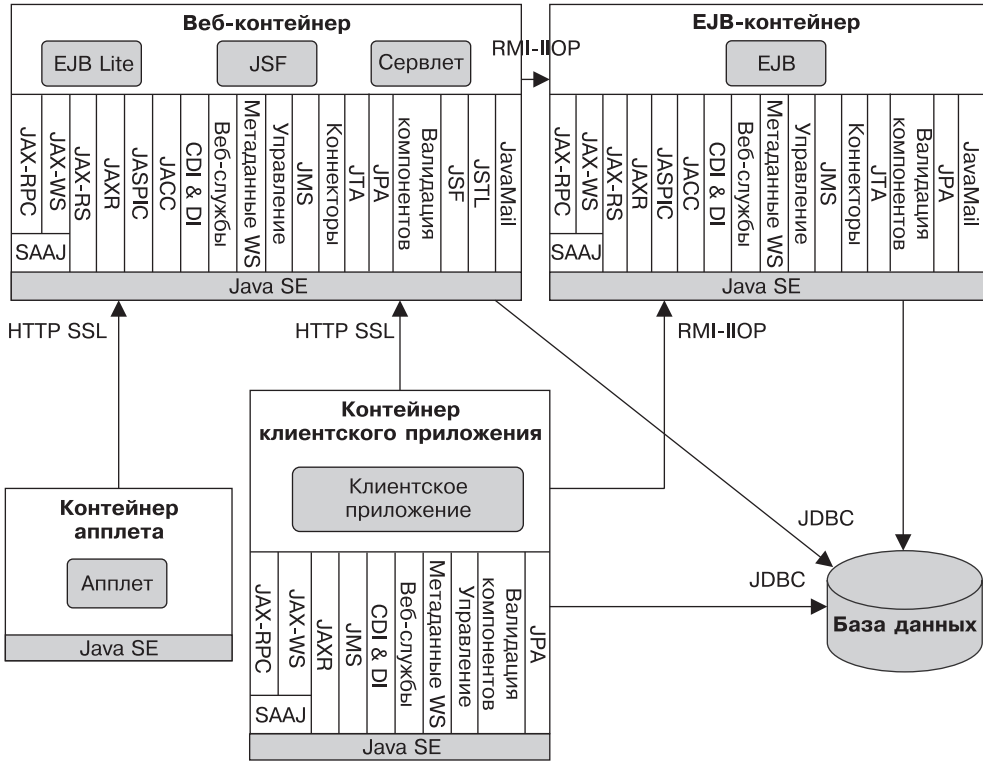


Рис. 1.2. Сервисы, предоставляемые контейнерами

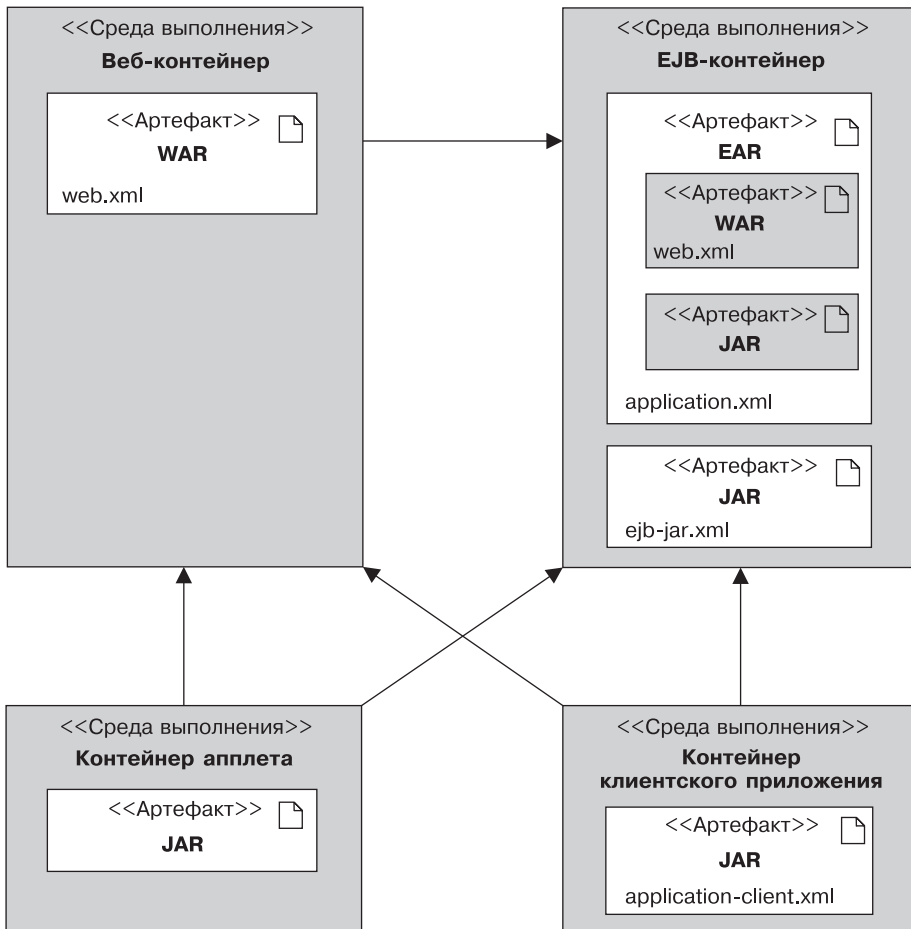
- ❑ **HTTP** — веб-протокол, повсеместно используемый в современных приложениях. В Java SE клиентский API определяется пакетом `java.net`. Серверный API для работы с HTTP определяется сервлетами, JSP-страницами, интерфейсами JSF, а также веб-службами SOAP и RESTful.
- ❑ **HTTPS** — представляет собой комбинацию HTTP и протокола безопасных соединений SSL.
- ❑ **RMI-IIOP** — удаленный вызов методов (RMI) позволяет вызывать удаленные объекты независимо от основного протокола. В Java SE нативным RMI-протоколом является протокол Java для удаленного вызова методов (JMRP). RMI-IIOP представляет собой расширение технологии RMI, которое используется для интеграции с архитектурой CORBA. Язык описания Java-интерфейсов (IDL) позволяет компонентам приложений Java EE вызывать внешние объекты CORBA с помощью протокола IIOP. Объекты CORBA могут быть написаны на разных языках (Ada, C, C++, Cobol и т. д.), включая Java.

## Упаковка

Для последующего развертывания в контейнере компоненты сначала необходимо упаковать в стандартно отформатированный архив. Java SE определяет файлы



архива Java (формат JAR), используемые для агрегации множества файлов (Java-классов, дескрипторов развертывания, ресурсов или внешних библиотек) в один сжатый файл (на основе формата ZIP). Как видно на рис. 1.3, Java EE определяет различные типы модулей, имеющих собственный формат упаковки, основанный на общем формате JAR.



**Рис. 1.3.** Архивы в контейнерах

- ❑ Модуль клиентских приложений содержит Java-классы и другие ресурсные файлы, упакованные в архив JAR. Этот файл может выполняться в среде Java SE или в контейнере клиентского приложения. Как любой другой архивный формат, JAR-файл содержит опциональный каталог **META-INF** для мета-информации, описывающей архив. Файл **META-INF/MANIFEST.MF** используется для определения данных, относящихся к расширениям и упаковке. При развертывании в контейнере клиентских приложений соответствующий дескриптор развертывания может быть опционально размещен по адресу **META-INF/application-client.xml**.

- ❑ Модуль EJB содержит один или несколько компонент-сеансов и/или компонентов, управляемых сообщениями (MDB), упакованных в архив JAR (часто называемый JAR-файл EJB). Он содержит опциональный дескриптор развертывания META-INF/ejb-jar.xml и может развертываться только в контейнере EJB.
- ❑ Модуль веб-приложений содержит сервлеты, страницы JSP и JSF, веб-службы, а также любые другие файлы, имеющие отношение к Сети (страницы HTML и XHTML, каскадные таблицы стилей (CSS), Java-сценарии, изображения, видео и т. д.). Начиная с Java EE 6 модуль веб-приложения также может содержать компоненты EJB Lite (подмножество интерфейса EJB API, описанное в главе 7). Все эти артефакты упаковываются в архив JAR с расширением WAR (также называемый архивом WAR или веб-архивом). Опциональный веб-дескриптор развертывания определяется в файле WEB-INF/web.xml. Если архив WAR содержит компоненты EJB Lite, то файл WEB-INF/ejb-jar.xml может быть снабжен опциональным дескриптором развертывания. Java-файлы с расширением .class помещаются в каталог WEB-INF/classes, а зависимые архивные JAR-файлы — в каталог WEB-INF/lib.
- ❑ Корпоративный модуль может содержать нуль или более модулей веб-приложений, модулей EJB, а также других общих или внешних библиотек. Они упаковываются в корпоративный архив (файл JAR с расширением .ear) таким образом, чтобы развертывание всех этих модулей происходило одновременно и согласованно. Опциональный дескриптор развертывания корпоративного модуля определяется в файле META-INF/application.xml. Специальный каталог lib используется для разделения общих библиотек по модулям.

## Аннотации и дескрипторы развертывания

В парадигме программирования существует два подхода: императивное и декларативное программирование. Первое устанавливает алгоритм для достижения цели (что должно быть сделано), тогда как второе определяет, как достичь цели (как это должно быть сделано). В Java EE декларативное программирование выполняется с помощью метаданных, а именно аннотаций и/или дескрипторов развертывания.

Как вы могли видеть на рис. 1.2, компоненты выполняются в контейнере, который, в свою очередь, дает компоненту набор сервисов. Метаданные используются для объявления и настройки этих сервисов, а также для ассоциирования с ними дополнительной информации, наряду с Java-классами, интерфейсами, конструкторами, методами, полями либо параметрами.

Начиная с Java EE 5, количество аннотаций в корпоративной платформе неуклонно растет. Они декорируют метаданными ваш код (Java-классы, интерфейсы, поля, методы). Листинг 1.1 показывает простой Java-объект в старом стиле (POJO), объявляющий определенное поведение с использованием аннотаций к классу и к атрибуту (подробнее о компонентах EJB, контексте хранения и аннотациях — в следующих главах).

**Листинг 1.1.** Компонент EJB с аннотациями

```
@Stateless
@Remote(ItemRemote.class)
@Local(ItemLocal.class)
```

**@LocalBean**

```
public class ItemEJB implements ItemLocal, ItemRemote {
    @PersistenceContext(unitName = "chapter01PU")
    private EntityManager em;
    public Book findBookById(Long id) {
        return em.find(Book.class, id);
    }
}
```

Второй способ объявления метаданных — использование дескрипторов развертывания. Дескриптор развертывания (DD) означает файл XML-конфигуратора, который развертывается в контейнере вместе с компонентом. Листинг 1.2 показывает дескриптор развертывания компонента EJB. Как и большинство дескрипторов развертывания в Java EE 7, он определяет пространство имен `http://xmlns.jcp.org/xml/ns/javaee` и содержит атрибут версии с указанием версии спецификации.

**Листинг 1.2.** Дескриптор развертывания компонента EJB

```
<ejb-jar xmlns="http://xmlns.jcp.org/xml/ns/javaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/ejb-jar_3_2.xsd"
        version="3.2" >

    <enterprise-beans>
        <session>
            <ejb-name>ItemEJB</ejb-name>
            <remote>org.agoncal.book.javaee7.ItemRemote</remote>
            <local>org.agoncal.book.javaee7.ItemLocal</local>
            <local-bean/>
            <ejb-class>org.agoncal.book.javaee7.ItemEJB</ejb-class>
            <session-type>Stateless</session-type>
            <transaction-type>Container</transaction-type>
        </session>
    </enterprise-beans>
</ejb-jar>
```

Для того чтобы дескрипторы развертывания учитывались при работе, они должны упаковываться вместе с компонентами в специальный каталог **META-INF** или **WEB-INF**. В табл. 1.1 приведен список дескрипторов развертывания Java EE и соответствующая спецификация (подробнее об этом читайте в следующих главах).

**Таблица 1.1.** Дескрипторы развертывания в Java EE0

| Файл                   | Спецификация | Местоположение       |
|------------------------|--------------|----------------------|
| application.xml        | Java EE      | META-INF             |
| application-client.xml | Java EE      | META-INF             |
| beans.xml              | CDI          | META-INF или WEB-INF |

Продолжение ↗

Таблица 1.1 (продолжение)

| Файл             | Спецификация          | Местоположение       |
|------------------|-----------------------|----------------------|
| ra.xml           | JCA                   | META-INF             |
| ejb-jar.xml      | EJB                   | META-INF или WEB-INF |
| faces-config.xml | JSF                   | WEB-INF              |
| persistence.xml  | JPA                   | META-INF             |
| validation.xml   | Валидация компонентов | META-INF или WEB-INF |
| web.xml          | Сервлет               | WEB-INF              |
| web-fragment.xml | Сервлет               | WEB-INF              |
| webservices.xml  | Веб-службы SOAP       | META-INF или WEB-INF |

Начиная с Java EE 5, большинство дескрипторов развертывания опциональны, и вместо них можно использовать аннотацию. Но вы также можете взять для вашего приложения наилучшее от обоих методов. Самое большое преимущество аннотаций в том, что они могут значительно сократить количество кода, который необходимо написать разработчику. Кроме того, используя аннотацию, вы можете избежать необходимости в дескрипторе развертывания. С другой стороны, дескрипторы развертывания — это внешние XML-файлы, для замены которых не требуется изменений исходного кода и recompilации. Если вы используете сразу оба метода, то во время развертывания приложения или компонента метаданные переопределяются дескриптором развертывания (то есть XML приоритетнее аннотаций).

#### ПРИМЕЧАНИЕ

Сегодня в Java-программировании предпочтительнее использование аннотаций, а не дескрипторов развертывания. Это происходит в рамках тенденции перехода от двуязычного программирования (Java + XML) к одноязычному (Java). Точно так же приложение проще анализировать и создавать его прототип, когда все (данные, методы и метаданные с аннотациями) хранится в одном месте.

Java EE использует понятие программирования путем исключения (также известное как программирование по соглашениям), когда большая часть общего поведения не требует сопровождения метаданными («в программировании метаданные являются исключением, контейнер заботится о настройках по умолчанию»). Это означает, что даже при малом количестве аннотаций или XML контейнер может выдать стандартный набор настроек с заданным по умолчанию поведением.

## Стандарты

Платформа Java EE основана на нескольких стандартах. Это означает, что Java EE проходит процесс стандартизации, принятый группой Java Community Process, и описывается в спецификациях. На самом деле Java EE объединяет несколько других спецификаций (или запросов на спецификацию Java), поэтому ее можно называть обобщающей. Вы можете спросить, почему стандарты так важны, если наиболее успешные Java-фреймворки не стандартизированы (Struts, Spring и т. д.).

На протяжении всей своей истории люди создавали стандарты, чтобы облегчить коммуникацию и обмен. В качестве наиболее выдающихся примеров можно привести язык, валюту, время, навигацию, системы измерений, инструменты, железные дороги, электричество, телеграф, телефонию, протоколы и языки программирования.

В самом начале развития Java, если вы занимались корпоративной или веб-разработкой, вы должны были подчиняться законам проприетарного мира, создавая свои собственные фреймворки, либо ограничивать себя проприетарным коммерческим фреймворком. Затем настало время свободных фреймворков, которые не всегда основываются на открытых стандартах. Вы можете использовать свободные фреймворки и ограничиваться одной-единственной реализацией либо применять такие открытые фреймворки, которые подчиняются стандартам, чтобы приложение было портируемым. Java EE предусматривает открытые стандарты, реализуемые несколькими коммерческими (WebLogic, Websphere, MQSeries и др.) или свободными (GlassFish, Jboss, Hibernate, Open JPA, Jersey и т. д.) фреймворками для работы с транзакциями, безопасностью, хранящими состояние компонентами, хранимостью объектов и т. д. Сегодня ваше приложение может быть развернуто на любом совместимом сервере приложений с очень малым количеством изменений.

## JCP

Java Communication Process — открытая организация, созданная в 1998 году компанией Sun Microsystems. Одно из направлений работы JCP — определение будущих версий и функционала платформы Java. Когда определяется необходимость в стандартизации существующего компонента или интерфейса, ее инициатор (руководитель спецификации) создает запрос на спецификацию Java (JSR) и формирует группу экспертов. Такие группы, состоящие из представителей компаний, организаций, университетов или частных лиц, ответственны за разработку запроса на спецификацию (JSR) и по итогу представляют:

- ❑ одну или несколько спецификаций, объясняющих детали и определяющих основные понятия запроса на спецификацию Java (JSR);
- ❑ *базовую реализацию (RI)*, которая является фактической реализацией спецификации;
- ❑ *пакет проверки совместимости* (известный также как *пакет проверки технологической совместимости*, или ТСК), представляющий собой набор тестов, которые должна пройти каждая реализация для подтверждения соответствия спецификации.

После одобрения исполнительным комитетом (ЕС) спецификация выпускается в Java-сообщество для внедрения.

## Портируемость

С самого создания целью Java EE было обеспечить разработку приложения и его развертывание на любом сервере приложений без изменения кода или конфигурационных файлов. Это никогда не было так просто, как казалось. Спецификации

не покрывают всех деталей, а реализации в итоге предоставляют непортируемые решения. Например, это случилось с именами JNDI. При развертывании компонента EJB на серверах GlassFish, Jboss или WebLogic на каждом из них было свое собственное имя для JNDI, так как в спецификации оно не было явно указано. В код приходилось вносить изменения в зависимости от используемого сервера приложений. Эта конкретная проблема была устранена в Java EE, когда установили синтаксис для имен JNDI.

Сегодня платформа представила наиболее портируемые параметры конфигурации, увеличив таким образом портируемость. Несмотря на то что некоторые API были признаны устаревшими (отсечены), приложения Java EE сохраняют полную совместимость с предыдущими версиями, позволяя вашему приложению перемещаться до более новых версий сервера приложений без особых проблем.

## Модель программирования

Большинство спецификаций Java EE 7 используют одну и ту же модель программирования. Обычно это POJO с некоторыми метаданными (аннотациями или XML), которые развертываются в контейнере. Чаще всего POJO даже не реализует интерфейс и не расширяет суперкласс. Благодаря метаданным контейнер знает, какие сервисы необходимо применить к этому развертываемому компоненту.

В Java EE 7 сервлеты, управляемые компоненты JSF, компоненты EJB, сущности, веб-службы SOAP и REST являются аннотированными классами с опциональными дескрипторами развертывания XML. Листинг 1.3 показывает управляемый компонент JSF, представляющий собой Java-класс с одиночной CDI-аннотацией.

**Листинг 1.3.** Управляющий компонент JSF

```
@Named
public class BookController {
    @Inject
    private BookEJB bookEJB;
    private Book book = new Book();
    private List<Book> bookList = new ArrayList<Book>();
    public String doCreateBook() {
        book = bookEJB.createBook(book);
        bookList = bookEJB.findBooks();
        return "listBooks.xhtml";
    }
    // Геттеры, сеттеры
}
```

Компоненты EJB следуют той же модели. Как показано в листинге 1.4, если вам нужно обратиться к компоненту EJB локально, для этого достаточно простого аннотированного класса без интерфейса. Компоненты EJB могут также развертываться напрямую в файл WAR без предварительного упаковывания в файл JAR. Благодаря этому компоненты EJB являются простейшими транзакционными сущностями и могут быть использованы как в сложных корпоративных, так и в простых веб-приложениях.

**Листинг 1.4.** Компонент EJB без сохранения состояния

```

@Stateless
public class BookEJB {
    @Inject
    private EntityManager em;
    public Book findBookById(Long id) {
        return em.find(Book.class, id);
    }
    public Book createBook(Book book) {
        em.persist(book);
        return book;
    }
}

```

Веб-службы RESTful широко распространились в современных приложениях. Спецификация JAX-RS для Java EE 7 была усовершенствована с учетом нужд больших предприятий. Как показано в листинге 1.5, веб-служба RESTful является аннотированным Java-классом, соответствующим действиям HTTP (подробнее читайте в главе 15).

**Листинг 1.5.** Веб-служба RESTful (с сохранением состояния представления)

```

@Path("books")
public class BookResource {
    @Inject
    private EntityManager em;
    @GET
    @Produces({"application/xml", "application/json"})
    public List<Book> getAllBooks() {
        Query query = em.createNamedQuery("findAllBooks");
        List<Book> books = query.getResultList();
        return books;
    }
}

```

В следующих главах вы будете периодически встречаться с кодом такого типа, где компоненты только содержат бизнес-логику, а метаданные представлены аннотациями (или XML). Таким образом гарантируется, что контейнер использует именно необходимые сервисы.

## Java Standard Edition 7

Важно подчеркнуть, что Java EE — это расширенная версия Java SE. Это означает, что в Java EE доступны все возможности языка Java, а также API.

Официальный релиз платформы Java SE 7 состоялся в июле 2011 года. Она была разработана в рамках запроса на спецификацию JSR 336 и предоставляла много новых возможностей, обеспечивая простоту разработки платформ предыдущих версий. Напомню, что функционал Java SE 5 включал автобоксинг, аннотирование, дженерики (обобщенные сущности), перечисление и т. д. Новинками Java SE 6 стали инструменты диагностирования, управления, мониторинга и интерфейс JMX API,

упрощенное выполнение сценарных языков на виртуальной машине Java. Java SE 7 объединяет запросы на спецификацию JSR 334 (чаще употребляется название Project Coin), JSR 292 (InvokeDynamic или поддержка динамических языков на виртуальной машине Java), JSR 203 (новый API ввода/вывода, обычно называемый NIO.2), а также несколько обновлений существующих спецификаций (JDBC 4.1 (JSR 221)). Даже если в этой книге не удастся подробно описать Java SE 7, некоторые из этих обновлений будут приведены в качестве примеров, поэтому я хочу предложить вам краткий обзор того, как они могут выглядеть.

## Строковый оператор

До появления Java SE 7 в качестве оператора ветвления могли использоваться только числа (типов `byte`, `short`, `int`, `long`, `char`) или перечисления. Теперь стало возможным применять переключатель с цифро-буквенными значениями `Strcompare`. Это помогает избежать длинных списков, начинающихся с `if/then/else`, и сделать код более удобочитаемым. Теперь вы можете использовать в своих приложениях код, показанный в листинге 1.6.

**Листинг 1.6.** Строковый оператор

```
Stringaction = "update";
switch (action) {
    case "create":
        create();
        break;
    case "read":
        read();
        break;
    case "update":
        update();
        break;
    case "delete":
        delete();
        break;
    default:
        noCrudAction(action);
}
```

## Ромбовидная нотация

Дженерики впервые появились в Java SE 5, но синтаксис их был достаточно простым. В Java SE 7 появился более лаконичный синтаксис, называемый ромбовидным. В таком варианте запись объявления объекта не повторяется при его инстанцировании. Листинг 1.7 содержит пример объявления дженериков как с применением ромбовидного оператора, так и без него.

**Листинг 1.7.** Объявление дженериков

```
// Без ромбовидного оператора
List<String> list = new ArrayList<String>();
Map<Reference<Object>, Map<Integer, List<String>>> map =
```



```
new HashMap<Reference<Object>, Map<Integer, List<String>>>());

// С ромбовидным оператором
List<String> list = new ArrayList<>();
Map<Reference<Object>, Map<Integer, List<String>>> map = new HashMap<>();
```

## Конструкция try-with-resources

В некоторых Java API закрытием ресурсов необходимо управлять вручную, обычно с помощью метода `close` в блоке `finally`. Это касается ресурсов, управляемых операционной системой, например файлов, сокетов или соединений интерфейса JDBC. Листинг 1.8 показывает, как необходимо ставить закрывающий код в блоке `finally` с обработкой исключений, но удобочитаемость кода из-за этого снижается.

**Листинг 1.8.** Закрытие входных/выходных потоков в блоках `Finally`

```
try {
    InputStream input = new FileInputStream(in.txt);
    try {
        OutputStream output = new FileOutputStream(out.txt);
        try {
            byte[] buf = new byte[1024];
            int len;
            while ((len = input.read(buf)) >= 0)
                output.write(buf, 0, len);
        } finally {
            output.close();
        }
    } finally {
        input.close();
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Конструкция `try-with-resources` решает проблему читаемости с помощью нового, более простого синтаксиса. Это позволяет ресурсам в блоке `try` автоматически высвобождаться в его конце. Нотация, описанная в листинге 1.9, может использоваться для любого класса, реализующего новый интерфейс `java.lang.AutoCloseable`. Сейчас он реализуется множеством классов (`InputStream`, `OutputStream`, `JarFile`, `Reader`, `Writer`, `Socket`, `ZipFile`) и интерфейсов (`java.sql.ResultSet`).

**Листинг 1.9.** Закрытие входных/выходных потоков с помощью конструкции `try-with-resources`

```
try (InputStream input = new FileInputStream(in.txt);
    OutputStream output = new FileOutputStream(out.txt)) {
    byte[] buf = new byte[1024];
    int len;
    while ((len = input.read(buf)) >= 0)
        output.write(buf, 0, len);
}
```

```
} catch (IOException e) {  
    e.printStackTrace();  
}
```

## Multicatch-исключения

До появления Java SE 6 блок захвата мог обрабатывать только одно исключение в каждый момент времени. Поэтому приходилось накапливать несколько исключений, чтобы потом применить нужное действие для обработки исключений каждого типа. И как показано в листинге 1.10, для каждого исключения зачастую необходимо выполнять одно и то же действие.

**Листинг 1.10.** Использование нескольких конструкций для захвата исключений

```
try {  
    // Какое-либо действие  
} catch (SAXException e) {  
    e.printStackTrace();  
} catch (IOException e) {  
    e.printStackTrace();  
} catch (ParserConfigurationException e) {  
    e.printStackTrace();  
}
```

Если разные исключения требуют одинаковой обработки, в Java SE 7 вы можете добавить столько типов исключений, сколько нужно, разделив их прямым следствием, как показано в листинге 1.11.

**Листинг 1.11.** Использование Multicatch-исключения

```
try {  
    // Какое-либо действие  
} catch ( SAXException | IOException | ParserConfigurationException e ) {  
    e.printStackTrace();  
}
```

## NIO.2

Если вам, как и многим Java-разработчикам, с трудом удастся читать или записывать некоторые файлы, вы оцените новую возможность Java SE 7: пакет ввода/вывода `java.nio`. Этот пакет, обладающий более выразительным синтаксисом, призван заменить существующий пакет `java.io`, чтобы обеспечить:

- ☐ более аккуратную обработку исключений;
- ☐ полный доступ к файловой системе с новыми возможностями (поддержка атрибутов конкретной операционной системы, символических ссылок и т. д.);
- ☐ добавление понятий `FileSystem` и `FileStore` (например, возможность разметки диска);
- ☐ вспомогательные методы (перемещение/копирование файлов, чтение/запись бинарных или текстовых файлов, путей, каталогов и т. д.).

В листинге 1.12 показан новый интерфейс `java.nio.file.Path` (используется для определения файла или каталога в файловой системе), а также утилитный класс

`java.nio.file.Files` (применяется для получения информации о файле или манипулирования им). Начиная с Java SE 7 рекомендуется использовать новый NIO.2, даже пока старый пакет `java.io` не вышел из употребления. В листинге 1.12 код получает информацию о файле `source.txt`, копирует его в файл `dest.txt`, отображает его содержимое и удаляет его.

**Листинг 1.12.** Использование нового пакета ввода/вывода

```
Path path = Paths.get("source.txt");
boolean exists = Files.exists(path);
boolean isDirectory = Files.isDirectory(path);
boolean isExecutable = Files.isExecutable(path);
boolean isHidden = Files.isHidden(path);
boolean isReadable = Files.isReadable(path);
boolean isRegularFile = Files.isRegularFile(path);
boolean isWritable = Files.isWritable(path);
long size = Files.size(path);
// Копирует файл
Files.copy(Paths.get("source.txt"), Paths.get("dest.txt"));
// Считывает текстовый файл
List<String> lines = Files.readAllLines(Paths.get("source.txt"), UTF_8);
for (String line : lines) {
    System.out.println(line);
}
// Удаляет файл
Files.delete(path);
```

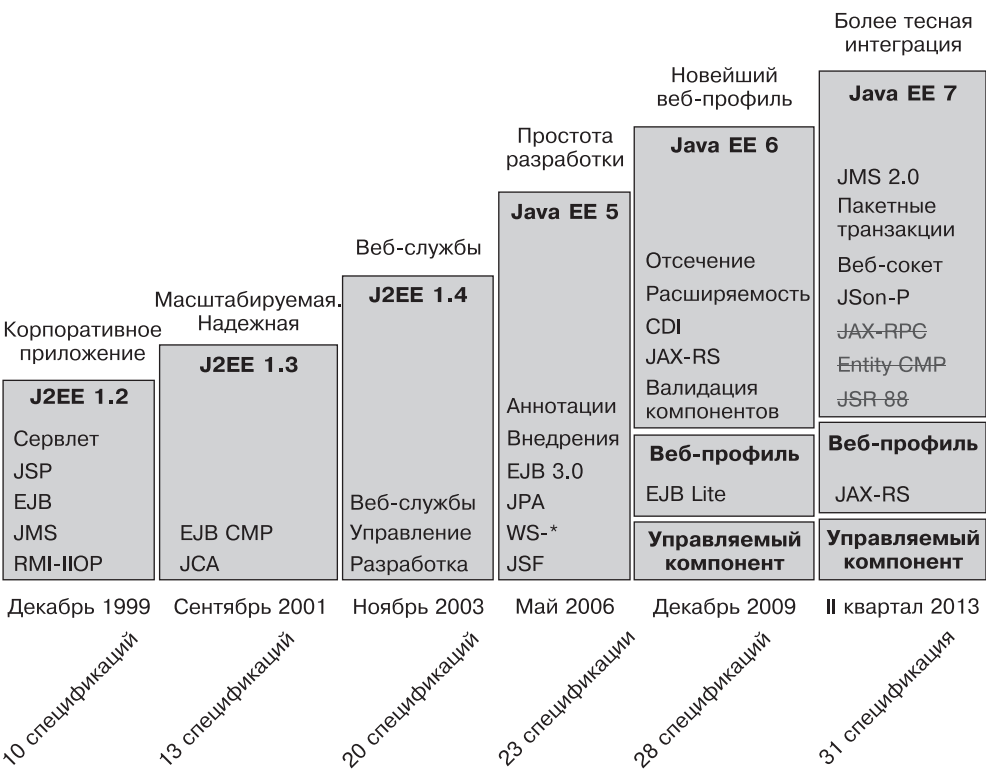
## Обзор спецификаций Java EE

Java EE — это обобщающая спецификация, которая объединяет и интегрирует остальные. На сегодняшний день для обеспечения совместимости с Java EE 7 сервер приложений должен реализовывать 31 спецификацию, а разработчику для оптимального использования контейнера необходимо знать тысячи API. Несмотря на то что требуется знать столько спецификаций и API, основная цель Java EE 7 — упростить платформу, предоставив несложную модель, основанную на POJO, веб-профиле и отсечении некоторых неактуальных технологий.

## Краткая история Java EE

На рис. 1.4 кратко изложена 14-летняя эволюция Java EE. Раньше Java EE называлась J2EE. Платформа J2EE 1.2 была разработана компанией Sun и впервые выпущена в 1999 году в качестве обобщающей спецификации, содержащей десять запросов JSR. В то время всеобщий интерес вызвала архитектура CORBA, поэтому J2EE была изначально нацелена на работу с распределенными системами. В ней уже существовала архитектура Enterprise Java Beans (EJB) с поддержкой удаленных служебных объектов как с сохранением состояния, так и без него и с возможностью поддержки хранимых объектов (компонентов-сущностей EJB). Они были основаны на транзакционной и распределенной компонентной модели, а в качестве базового протокола использовали RMI-IIOP (протокол удаленного вызова методов

и обмена между ORB в Интернете). Веб-уровень содержал сервлеты и страницы JSP, а для сетевой коммуникации использовалась служба сообщений Java (JMS).



**Рис. 1.4.** История J2EE/Java EE

Начиная с J2EE 1.3, разработкой спецификаций занималась группа Java Community Process (JCP) в рамках запроса JSR 58. Поддержка компонентов-сущностей стала обязательной, а в компонентах EJB появились дескрипторы развертывания XML для хранения метаданных (которые были сериализованы в отдельном файле в EJB 1.0). В этой версии была решена проблема издержек, связанных с передачей аргументов по значению при использовании удаленных интерфейсов. Для ее устранения появились локальные интерфейсы и возможность передачи аргументов по ссылке. Была разработана архитектура JCA (J2EE Connector Architecture), позволившая связать Java EE с корпоративной информационной системой (EIS).

**ПРИМЕЧАНИЕ**

CORBA была разработана в 1988 году именно потому, что корпоративные системы становились распределенными (примеры таких систем: Tuxedo и CICS). Затем последовали EJB и J2EE, которые создавались на десять лет позже, но также предназначались для работы с распределенными системами. К моменту разработки J2EE CORBA имела основательную поддержку и являлась промышленным ПО, однако компании нашли более